

Cerebras Confidential Computing Architecture (C-CCA)

Technical research report and proposed platform design

Confidential VMs, wafer-scale TEEs, protected AI memory, remote attestation, cryptographic key isolation, and Rust/Mojo software access

Purpose. This report proposes a hardware-plus-software confidential-computing architecture for future Cerebras wafer-scale systems, intended to protect AI models, sensitive data, and cryptographic material during inference, fine-tuning, and deep-learning workloads. It is written as an architecture/research proposal, not as a claim that the current WSE-3T or CS-4 already exposes the full TEE feature set described here.

Scope. The design covers CPU confidential VMs (CVMs), a WSE-native confidential execution domain, on-wafer SRAM and fabric isolation, secure I/O and MemoryX/SwarmX paths, measured boot, remote attestation, KMS/HSM-controlled key release, non-exportable cryptographic handles, threshold cryptography, and a software stack centered on Rust and Mojo. Existing Cerebras PyTorch/Python workflows are treated as compatibility layers and developer UX rather than the preferred minimal trusted computing base.

Contents

Executive summary

1. Strategic context: why confidential Cerebras matters
2. Cerebras architectural baseline
3. Security model and assumptions
4. C-CCA reference architecture
5. WSE hardware security architecture
6. Memory, fabric, DMA, and I/O protection
7. Secure boot, measurement, and remote attestation
8. Cryptographic architecture and private-key protection
9. Confidential LLM inference
10. Confidential training and fine-tuning
11. Rust and Mojo software architecture
12. API and compiler integration proposal
13. Multi-WSE, MemoryX, SwarmX, and cluster confidentiality
14. Side-channel, rollback, and operational security
15. Performance and implementation considerations
16. Roadmap and validation program
17. Demand and high-value application domains
18. Standards and interoperability strategy
19. Strategic recommendation to Cerebras
20. Research questions and next steps

Appendix A. Security invariants

Appendix B. Threat matrix

Appendix C. Terminology

Sources

Executive summary

The core proposal is to extend Cerebras from secure AI infrastructure into a remotely attested confidential AI platform. A conventional confidential VM protects CPU memory from an untrusted hypervisor, but a Cerebras workload also depends on the WSE, its distributed SRAM, wafer-scale fabric, DMA/I/O paths, and—under weight streaming—external model storage and transport. Therefore the unit of trust should be the complete AI workload, not the host server or CPU VM alone.

C-CCA defines a composite trusted execution environment composed of four cooperating domains: (1) a CPU CVM that owns policy, orchestration, attestation and secret brokering; (2) a WSE Confidential Domain that enforces hardware isolation over processing elements, local SRAM, the wafer fabric and device interfaces; (3) a protected model/state path covering MemoryX and relevant I/O or interconnect; and (4) an external attestation and key authority, normally a customer-controlled or independently operated KMS/HSM stack.

The architecture is motivated by the direction of the Cerebras platform itself. Cerebras announced CS-4 in August 2026 as a rack-scale system based on three WSE-3 Turbo processors; the WSE-3T is described as having four trillion transistors, 900,000 AI-optimized cores, 44 GB of on-wafer SRAM, 250 PFLOPS of compute, and 43.2 PB/s of memory bandwidth. Cerebras also emphasizes low-latency wafer I/O and agentic inference as strategic workloads.

The proposal is therefore intentionally hardware-native. Rather than cloning SGX page enclaves, C-CCA uses coarse-grained spatial confidential domains across the wafer. A domain receives a cryptographically protected Domain ID, domain-specific memory/fabric keys, and access rights that are enforced by the memory and network fabric. The approach maps the security boundary onto Cerebras' architecture: PE + local SRAM + fabric + DMA + device I/O form one protected execution region.

The cryptographic design treats the TEE as the last-resort plaintext residence for secrets. Long-lived root authority should remain in HSMs/KMSs or threshold infrastructure; when a private key, DEK, threshold share, or session key must be in plaintext to perform an operation, it is released only to attested code inside the protected domain. APIs expose non-exportable handles such as `sign()`, `decrypt()`, `derive()`, and `threshold_sign()` instead of returning private-key bytes to general software.

For software, Rust and Mojo can form a modern security/AI systems layer without requiring either language to replace the entire Cerebras stack immediately. Rust should own the security control plane, attestation agent, secret broker, policy engine, lifecycle, IPC, and cryptographic service. Mojo should be the preferred systems/AI kernel language for performance-sensitive secure device code and can exploit MLIR and LLVM-level extensibility. Rust and Mojo have different compiler IR paths—Rust uses MIR before lowering to LLVM IR, while Mojo is built around MLIR and lowers through LLVM-level representations—so the proposal should not describe them as sharing a single MIR/MLIR pipeline.

The recommended path is software-first and silicon-forward: build the attestation/key-release and CVM prototype around current Cerebras systems; then define a device-level secure channel and workload measurement; then introduce a WSE root of trust, domain isolation, protected SRAM/fabric and secure DMA in a future WSE generation; finally extend confidentiality across MemoryX, SwarmX and multi-WSE clusters. This produces an early customer story without waiting for new silicon while keeping the ultimate security boundary architecturally coherent.

Design objective	Proposed C-CCA mechanism	Security outcome
Protect CPU-side secrets	TDX/SEV-SNP CVM + Rust secret broker	Host/hypervisor cannot directly read CVM memory
Protect WSE execution	WSE Confidential Domain + Domain ID	Hardware boundary around PE/SRAM/fabric execution
Protect model state	Per-domain encryption + integrity + secure model path	Weights, activations and KV cache remain protected
Protect private keys	Non-exportable handles + TEE crypto service	Plaintext key bytes stay inside protected memory
Prove platform state	Composite measured boot + remote attestation	Relying party can verify hardware/software state
Control key release	Attestation result + policy + KMS/HSM	Keys released to approved code, not machines
Protect multi-WSE jobs	Composite cluster evidence	All members of a confidential workload are attestable
Enable modern software	Rust control plane + Mojo secure AI kernels	Memory-safe systems code + MLIR-based AI compiler path

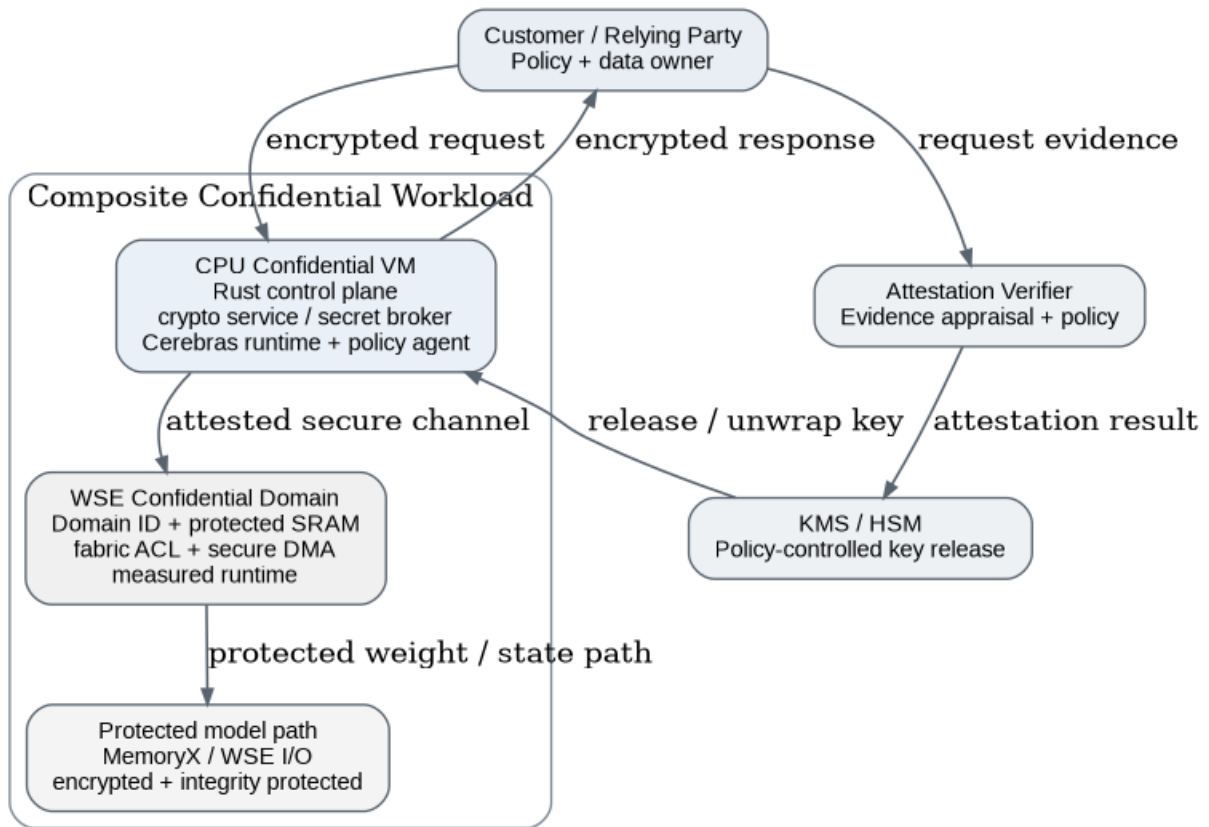


Figure 1. C-CCA reference architecture: the confidential workload is the CPU CVM + WSE confidential domain + protected model path, with external attestation and KMS/HSM policy enforcement.

1. Strategic context: why confidential Cerebras matters

Confidential computing closes a specific gap in conventional security architecture: the period when data and code are decrypted for execution. Encryption at rest protects stored artifacts and TLS protects network transport, but neither inherently prevents a privileged host, hypervisor or management plane from inspecting plaintext state while a workload executes. Modern confidential-computing platforms address this by combining hardware isolation, memory protection and attestation. Intel TDX uses hardware-isolated domains, memory-encryption mechanisms and remote attestation; AMD SEV similarly provides encrypted VM memory and attestation, with newer generations extending the boundary to compatible devices.¹²

AI increases the value of that protection because the workload simultaneously carries multiple classes of high-value secrets: model weights and adapters, prompts and user data, RAG context, intermediate activations, KV cache, optimizer state, API credentials, signing keys and cryptographic session material. Microsoft's Confidential AI guidance explicitly frames confidential computing as protection of models and data across the AI lifecycle, including training, inference and collaborative processing.³

The market signal is strong enough to justify a platform-level response. The Confidential Computing Consortium's 2025 industry study reported full-production deployments of 37% in financial services, 29% in healthcare and 21% in government, while healthcare respondents placed particularly high priority on privacy-preserving multi-party collaboration.⁴

1 Intel, "Intel Trust Domain Extensions (Intel TDX)."

2 AMD, "Confidential Computing Starts with AMD EPYC Server CPUs."

3 Microsoft Learn, "Confidential AI - Azure Confidential Computing."

4 Confidential Computing Consortium, "2025 industry study / adoption findings."

Cerebras already has an unusually strong performance proposition for applications where inference latency is part of the user experience. CS-4 is explicitly positioned around agentic workloads; Cerebras states that lower token latency leaves room for more reasoning, verification and tool use in an agentic system.⁵ That creates a direct security implication: the faster the model can reason and call tools, the more valuable it becomes to confine the model and its credentials inside an attested execution boundary.

A confidential Cerebras platform should therefore be understood as a complement to, not a replacement for, the existing performance architecture. The commercial proposition is not simply "encrypted memory." It is "cryptographically verifiable AI execution at wafer scale."

2. Cerebras architectural baseline

Cerebras' architecture is materially different from a conventional CPU + GPU server. The public Cerebras software documentation describes a Wafer-Scale cluster with WSE systems, MemoryX for model weights, SwarmX for connecting multiple systems, input preprocessing servers, and management servers. In current documented workflows, user nodes submit jobs and cluster management software orchestrates the execution environment.⁶

The newer CS-4 architecture keeps the wafer-scale model but changes the system packaging and I/O. Cerebras states that CS-4 contains three WSE-3 Turbo processors, each housed in a modular compute backpack, with programmable wafer I/O and very low wafer-to-wafer latency. The published WSE-3T figures are 900,000 AI-optimized cores, 44 GB SRAM, 250 PFLOPS and 43.2 PB/s memory bandwidth.⁷

Cerebras also continues to support weight streaming as a way to separate large model-parameter storage from compute. In the documented architecture, MemoryX stores model weights and streams them layer-by-layer into the WSE; SwarmX broadcasts weights across systems and reduces gradients for distributed training.⁸

This architecture yields three confidentiality boundaries that must be considered separately: the host/CPU boundary, the WSE device boundary, and the model-state/network/storage boundary. A design that protects only CPU DRAM may still leave plaintext model state exposed in device memory or on a management/data path. Likewise, a design that protects the WSE but leaves MemoryX or network staging outside the trust boundary can leak model weights or training data.

Cerebras element	Role in existing architecture	C-CCA security treatment
CPU / management nodes	User control, orchestration, preprocessing	Place sensitive control plane in CVM; minimize host TCB
WSE / WSE-3T	Core AI execution + distributed local memory	Create attested WSE Confidential Domains
On-wafer SRAM	Model/state/activation storage close to compute	Per-domain encryption + integrity + access control
Wafer-scale fabric	Connects distributed compute elements	Domain-aware routing + authenticated packets
MemoryX	Large-scale model weight storage / streaming	Encrypt at rest and in transit; key release after attestation
SwarmX	Cross-WSE weights/gradients data movement	Treat as protected inter-domain transport
I/O / external links	Host and cluster connectivity	Authenticated/encrypted device sessions + DMA controls
Management/runtime	Schedules and manages jobs	Untrusted by default; privileged but outside tenant TEE

⁵ Cerebras Systems, CS-4 announcement and discussion of agentic inference (18 Aug 2026).

⁶ Cerebras Developer Documentation, "Overview / Cerebras Wafer-Scale cluster."

⁷ Cerebras Systems, "Cerebras Unveils CS-4: Up to 30 Times Faster than GPU-based Solutions" (18 Aug 2026).

⁸ Cerebras Systems, "Announcing the Cerebras Architecture for Extreme-Scale AI."

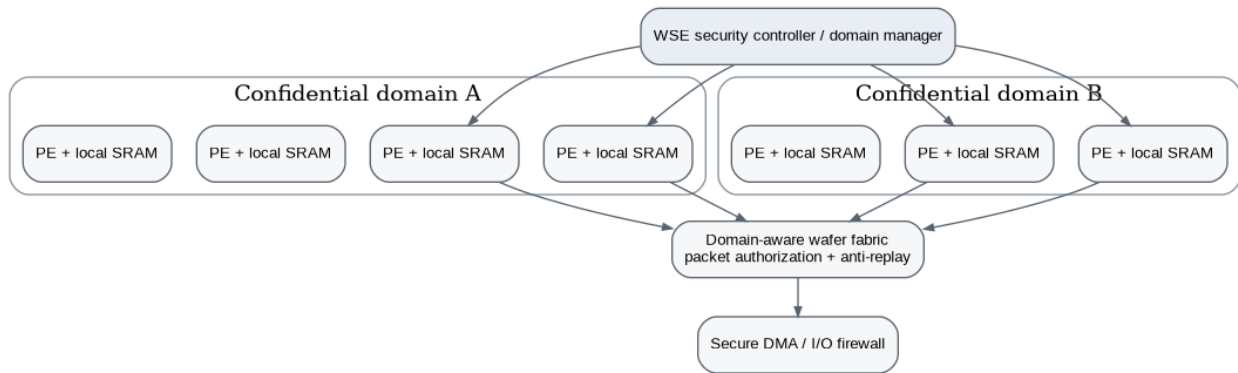


Figure 2. Proposed WSE Confidential Domain. The security boundary is coarse-grained and spatial rather than one enclave per processing element.

3. Security model and assumptions

C-CCA starts by defining what it protects from. The baseline adversary is a privileged infrastructure operator who may control the host OS, hypervisor, management software, drivers, storage fabric, and ordinary device-management interfaces. The architecture is not intended to make physical possession of the machine irrelevant; instead, it aims to protect against software-level privileged access and a defined set of physical and side-channel threats.

The threat model should explicitly separate four levels. Level 1 covers host memory inspection, malicious hypervisor behavior, unauthorized DMA, malicious management software and firmware substitution. Level 2 extends to cross-domain fabric interference, replay/rollback, debug abuse and device-interface attacks. Level 3 considers timing and contention leakage. Level 4 is high-assurance physical attack resistance, including probing, fault injection, power analysis and electromagnetic analysis. Each level should be a separately priced or certified security profile rather than an ambiguous "TEE" claim.

The security target is therefore a set of invariants. A valid deployment should guarantee that secret release requires valid attestation; confidential execution memory is not readable by the host; plaintext model state does not leave the trusted domain; private-key material is not exportable through the crypto service; fabric and DMA access cannot cross domains; and domain destruction invalidates the operational key hierarchy.

Asset	Primary exposure	Required control
LLM weights / adapters	Host RAM, MemoryX, staging buffers	Encrypted storage + attested key release + protected WSE path
Prompt / PHI / PII	Host preprocessing, logs, buffers	CVM/WSE confidentiality + logging policy
Activations / KV cache	WSE SRAM / diagnostic interfaces	Domain isolation + encrypted SRAM + debug gating
Optimizer state / gradients	Training memory + checkpoint path	Protected training domain + encrypted checkpoints
Private keys / threshold shares	Process memory / host inspection	Non-exportable handles in crypto TEE
Attestation private key	Forged evidence	Hardware-protected key never exposed to OS
Firmware / runtime	Supply-chain substitution	Secure + measured boot + version policy
Cluster topology/state	Cross-job interference	Composite attestation + domain membership

4. C-CCA reference architecture

The reference design has five planes: a hardware security plane, a confidential execution plane, a protected data plane, an attestation/policy plane, and a software/developer plane. This division is useful because different Cerebras teams can implement the planes incrementally while preserving the same external contract.

The hardware security plane contains the device root of trust, secure boot controller, domain manager, memory protection engine, fabric access-control logic, secure DMA, and debug gate. The confidential execution plane contains the CPU CVM and WSE domains. The data plane contains model weights, user data, activations, KV cache, optimizer

state and checkpoint streams. The attestation/policy plane sits outside the workload and is intentionally capable of being customer-controlled. The software plane exposes these capabilities through stable Rust and Mojo APIs.



Figure 3. Trust establishment and key-release chain. The key property is that attestation is a policy decision, not merely a hardware self-report.

The attestation model should follow the general architecture of IETF RATS: an Attester produces Evidence, a Verifier appraises it against policy and supply-chain reference values, and a Relying Party uses the resulting Attestation Result to make a resource-access decision. RFC 9334 explicitly includes confidential machine-learning model protection and describes the pattern in which a verifier can release a model or a key after successful appraisal.⁹

For device-to-host protocol design, SPDm is a useful building block. NVIDIA uses SPDm sessions to bind a GPU to the CPU confidential environment, and PCI-SIG TDISP defines a model for securely composing a virtualized TEE with a device interface. Those standards are not a complete Cerebras solution, but they provide a vocabulary and interoperability target for device authentication, measurement exchange and protected I/O.¹⁰¹¹

5. WSE hardware security architecture

The central new silicon block is a WSE Security Manager. It should own device identity, boot verification, domain lifecycle, per-domain keys, security state, and attestation evidence. The WSE Security Manager should be independent of the tenant runtime and have access to hardware state that software cannot forge, including the secure boot result, security-version state, domain assignments and protected memory/fabric configuration.

A Hardware Root of Trust should anchor the device identity and the first stage of the boot chain. The root need not expose a conventional general-purpose private key to firmware. Instead, a device-specific protected key hierarchy can derive an attestation signing identity, domain keys and other internal secrets under policy. The design should support revocation and security-version counters so that a compromised firmware revision can be excluded without changing the physical root key.

The WSE security model should use a Confidential Domain ID (CDID) as a first-class hardware metadata field. Each processing element, SRAM region, router/fabric endpoint, DMA channel and protected external transaction is associated with a CDID and an access policy. This is the foundation for spatial isolation: the WSE does not need hundreds of thousands of SGX-style enclaves; it needs a small number of large, hardware-enforced security domains that match real AI workloads.

A domain manager should support lifecycle states such as CREATED, MEASURED, KEYED, ACTIVE, SUSPENDED, QUARANTINED and DESTROYED. Key material associated with the domain should only transition from a sealed state to an operational state after the attestation/policy handshake. DESTROYED should zeroize local state and irreversibly invalidate the operational key lineage.

Primitive	Proposed behavior	Implementation note
Root of trust	Immutable measurement and device identity anchor	Can use a dedicated security controller / secure element style block
Secure boot	Verify signed security firmware and runtime before normal execution	Measure every stage; fail closed for confidential mode
CDID	Associate compute/memory/fabric objects with a confidential domain	Hardware enforced, not only software metadata
Domain keys	Derive per-domain keys from device root + domain nonce/measurement	Keys never exported as raw bytes
Domain manager	Create, bind, attest, activate, suspend, destroy domains	Minimal TCB and separate from general runtime
Debug gate	Disable or strongly gate production debug when domain active	Authorization should itself be attested and audited
Security version	Monotonic anti-rollback state	Bind to firmware and workload policy

⁹ IETF RFC 9334, “Remote ATtestation procedureS (RATS) Architecture.”

¹⁰ DMTF, “Security Protocol and Data Model (SPDM).”

¹¹ PCI-SIG, “TEE Device Interface Security Protocol (TDISP).”

6. Memory, fabric, DMA, and I/O protection

Memory confidentiality must be accompanied by access control and integrity. A simple encryption engine that allows an untrusted component to request plaintext reads is not a TEE. The design needs a hardware path in which the security domain is determined before decryption, and in which integrity/version information is checked before sensitive data becomes usable by the compute element.



Figure 4. Proposed protected memory/fabric transaction path. Encryption is applied only after authorization; fabric packets also carry domain and replay-protection context.

A practical design is to attach a small metadata tuple to protected memory transactions: CDID, KeyID, permission class and a monotonic or sequence-related integrity value. On access, the hardware checks domain membership and capability rights, derives/selects the appropriate data-encryption context, verifies the authenticated integrity state, and only then presents plaintext to the permitted PE. The exact cryptographic primitive can be tuned to Cerebras bandwidth and area constraints; the architectural invariant is that encryption and authorization are inseparable.

Because the wafer-scale fabric is itself part of the computation, fabric security needs more than endpoint memory ACLs. Routers and links should enforce destination/domain policy, reject packets whose source domain is not authorized for the destination, and provide replay/sequence protection for traffic that carries confidential state. A compromised software component should not be able to forge a packet that causes another confidential domain to receive plaintext or execute unauthorized commands.

DMA and I/O need an equivalent treatment. The WSE should expose a secure-DMA mode in which the DMA engine accepts transfers only for an attested domain and only to buffers whose ownership is recognized by the security manager. If the interface is physically attached to a host or PCIe-like fabric, the architecture should use protected device-interface semantics rather than trusting the host driver. TDISP is a useful reference model for this category of problem.¹²

For MemoryX and other disaggregated storage paths, the system should encrypt model weights outside the protected domain and establish a session key only after the target WSE domain has been attested. The plaintext path should terminate at a hardware-controlled ingress into the confidential domain rather than at a host-visible bounce buffer.

7. Secure boot, measurement, and remote attestation

The device attestation evidence should be composable. At minimum, the final evidence bundle should cover WSE hardware identity, firmware measurements, security-configuration state, domain configuration, runtime measurement, workload/application measurement, model or image measurement, security version, and a freshness value or nonce. This is deliberately richer than a static device certificate because the relying party needs to know what is actually running.

The preferred protocol layering is: secure boot creates the measured state; the WSE Security Manager exposes evidence; SPDH or an equivalent device protocol establishes an authenticated session; the CPU CVM and WSE exchange evidence; the external Verifier appraises the bundle; the KMS/HSM decides whether to release workload keys. The WSE should never become the final policy authority for customer secrets; the customer or tenant should remain in control of the policy that determines whether the measured state is acceptable.

A critical extension is workload measurement. The platform should support a stable manifest containing hashes or signed identifiers for the inference/training runtime, security policy, model image, tokenizer/graph image where material, and selected configuration parameters. The resulting measurement does not have to hash every transient tensor; instead it establishes that the software which will handle those tensors is the expected software.

The same model should extend to a multi-WSE cluster. IETF RATS explicitly discusses composite devices and layered attestation, which maps well to Cerebras systems in which a single workload may span multiple compute components and supporting devices.¹³

¹² PCI-SIG, “TEE Device Interface Security Protocol (TDISP).”

¹³ IETF RFC 9334, “Remote ATtestation procedureS (RATS) Architecture.”

The design should support both remote and air-gapped verification. Remote services are convenient for public-cloud deployments; local or customer-controlled verification is important for sovereign, defense and high-assurance deployments. NVIDIA already supports both remote attestation and local verification for its confidential GPU stack, demonstrating the operational value of this split.¹⁴

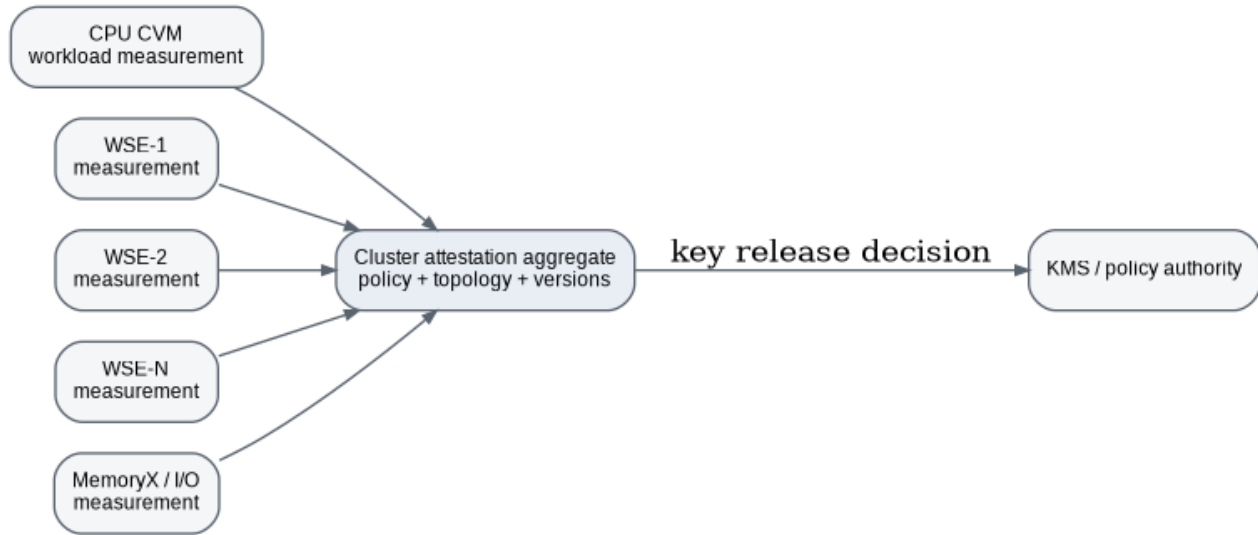


Figure 5. Composite cluster attestation. Every WSE and relevant supporting component contributes evidence to one workload-level trust decision.

8. Cryptographic architecture and private-key protection

The cryptographic layer should enforce a simple rule: plaintext key material may exist only inside the smallest trusted code and memory domain that needs it, for the shortest practical lifetime. The KMS/HSM remains the durable authority. The TEE is an execution environment for operations, not the permanent storage location of every master key.

¹⁴ NVIDIA, “Confidential Computing on NVIDIA H100 GPUs for Secure and Trustworthy AI.”

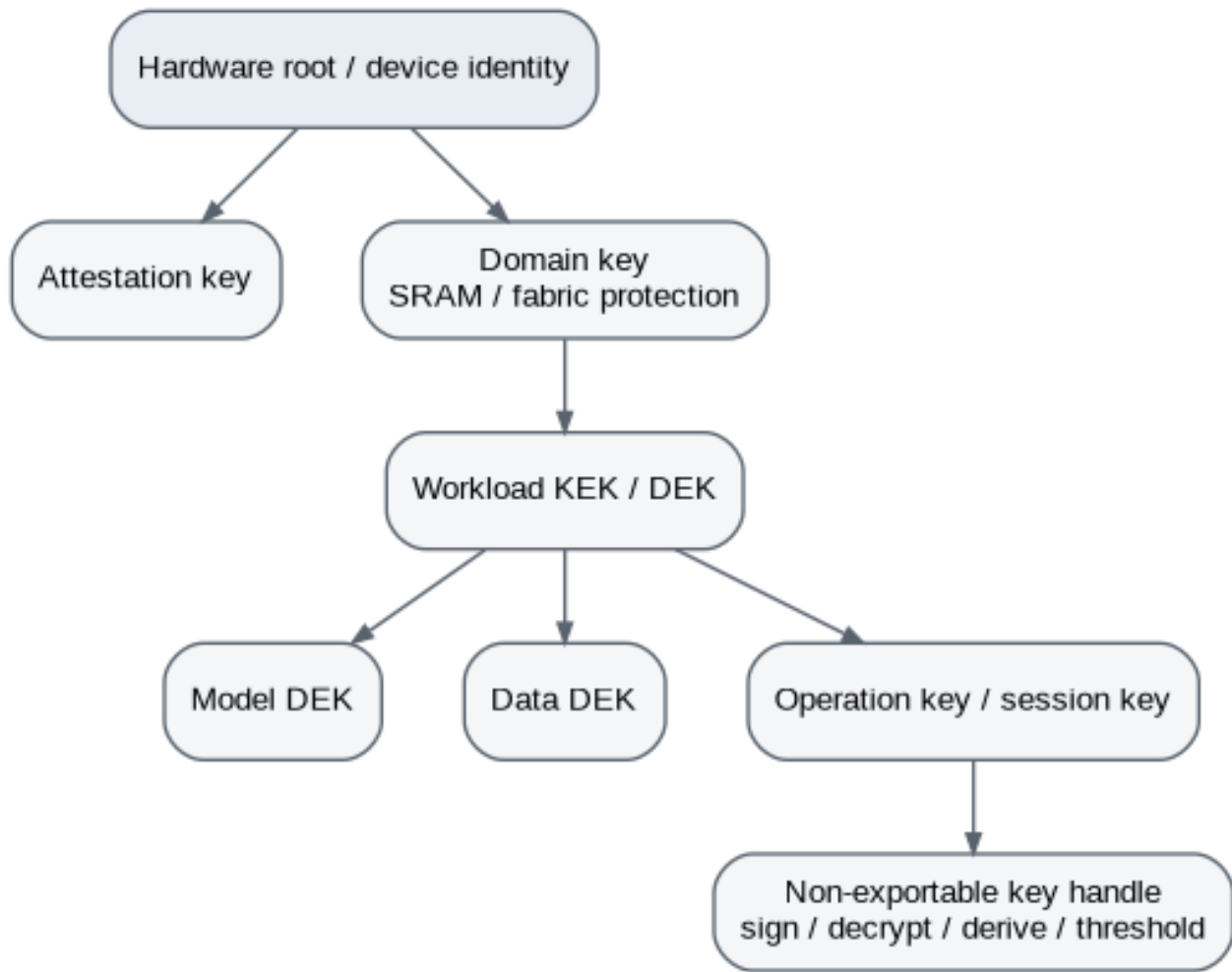


Figure 6. Proposed key hierarchy. Long-lived root authority stays outside the workload; operational key material is derived and used inside attested domains.

The service interface should be handle-based. An application receives a key handle or capability, not a byte array. Operations include symmetric encryption/decryption, public-key signing, signature verification, key derivation, key wrapping/unwrapping, and threshold partial operations. The interface should record the key purpose, allowed algorithm family, context, expiry/epoch, and attestation policy. This makes the cryptographic authority auditable and helps prevent an LLM agent from converting a general signing credential into an unrestricted secret-export primitive.

For symmetric cryptography, the platform should support high-throughput AEAD designs suitable for model and data encryption. AES-GCM is a natural hardware candidate; ChaCha20-Poly1305 is valuable as a software-implementation option. For memory encryption specifically, the exact mode should be selected with careful attention to parallelism, tweak/counter construction, integrity metadata, and random-access semantics. The system should expose algorithm agility rather than making one primitive part of the long-term architectural contract.

For asymmetric and threshold cryptography, the key should never be exported. For example, an ECDSA/Ed25519/BLS-style service can accept a digest and return a signature; a threshold service can accept a transaction digest and return a partial signature or a completed threshold signature. In a strong deployment, multiple threshold participants should also have independently verifiable attestations before they are allowed to perform the joint operation.

Zeroization must be designed as a system property, not merely a library convention. Rust has mature crates such as zeroize that provide compiler-resistant memory clearing, but software zeroization is only one layer. The hardware domain manager should also destroy the domain key lineage and scrub protected buffers on lifecycle termination.¹⁵

¹⁵ RustCrypto, “zeroize” crate documentation.

A conceptual Rust interface could look like this:

```
pub trait ConfidentialCrypto {
    fn import_sealed(&self, blob: &[u8], policy: KeyPolicy) -> Result<KeyHandle>;
    fn encrypt(&self, key: KeyHandle, aad: &[u8], plaintext: &[u8]) -> Result<Ciphertext>;
    fn decrypt(&self, key: KeyHandle, aad: &[u8], ciphertext: &[u8]) -> Result<Plaintext>;
    fn sign(&self, key: KeyHandle, digest: &[u8]) -> Result<Signature>;
    fn derive(&self, parent: KeyHandle, context: &[u8]) -> Result<KeyHandle>;
    fn threshold_sign(&self, policy: ThresholdPolicy, digest: &[u8]) -> Result<Signature>;
}
```

9. Confidential LLM inference

Confidential inference is the first flagship workload because it combines strong data sensitivity with the commercial advantages of Cerebras latency. The minimum confidentiality set should include data model weights, system prompt, user prompt, retrieved context, activations, KV cache, tool credentials and output before encryption. The exact set depends on the deployment, but the architecture should make these items configurable under one workload policy.



Figure 7. Confidential inference flow. Keys are released only after the CPU + WSE + workload state has passed policy appraisal.

For a model that fits primarily in on-wafer memory, the secure loading path should minimize plaintext residency on the host: retrieve an encrypted model image, attest the target domain, unwrap the model key inside the protected environment, and inject plaintext model state directly into protected WSE memory. For weight-streamed models, the same idea extends to MemoryX: encrypted weights remain protected until secure ingress and are decrypted into protected device state.

The KV cache deserves first-class treatment. In long-context and agentic workloads it can hold a large amount of sensitive conversational or retrieved context and may persist for longer than a single token. A confidential design should treat the KV cache as tenant secret state, subject to domain memory protection, migration policy, checkpoint policy and destruction rules.

Logging and observability must be security-aware. A standard production stack may log prompts, tokens, exceptions and performance data. In C-CCA, logs should default to metadata-only, with explicit policy gates for any content-bearing diagnostics. Performance counters and debug modes should be treated as part of the attack surface because they can enable side channels; NVIDIA, for example, disables performance counters in full H100 confidential mode.¹⁶

10. Confidential training and fine-tuning

Training and fine-tuning are more difficult than inference because the protected state is larger and more dynamic. In addition to model weights, a training TEE may need to protect gradients, activations, optimizer state, scheduler state, adapter weights, random seeds, checkpoints and portions of the data pipeline. The good news is that Cerebras already supports training and fine-tuning workflows and has documented LLM fine-tuning on its systems, so C-CCA can be introduced as a security layer around an existing execution model rather than inventing a new training abstraction.¹⁷

¹⁶ NVIDIA, H100 confidential-computing operation modes and security controls.

¹⁷ Cerebras Developer Documentation, “Training and fine-tuning a Large Language Model (LLM).”

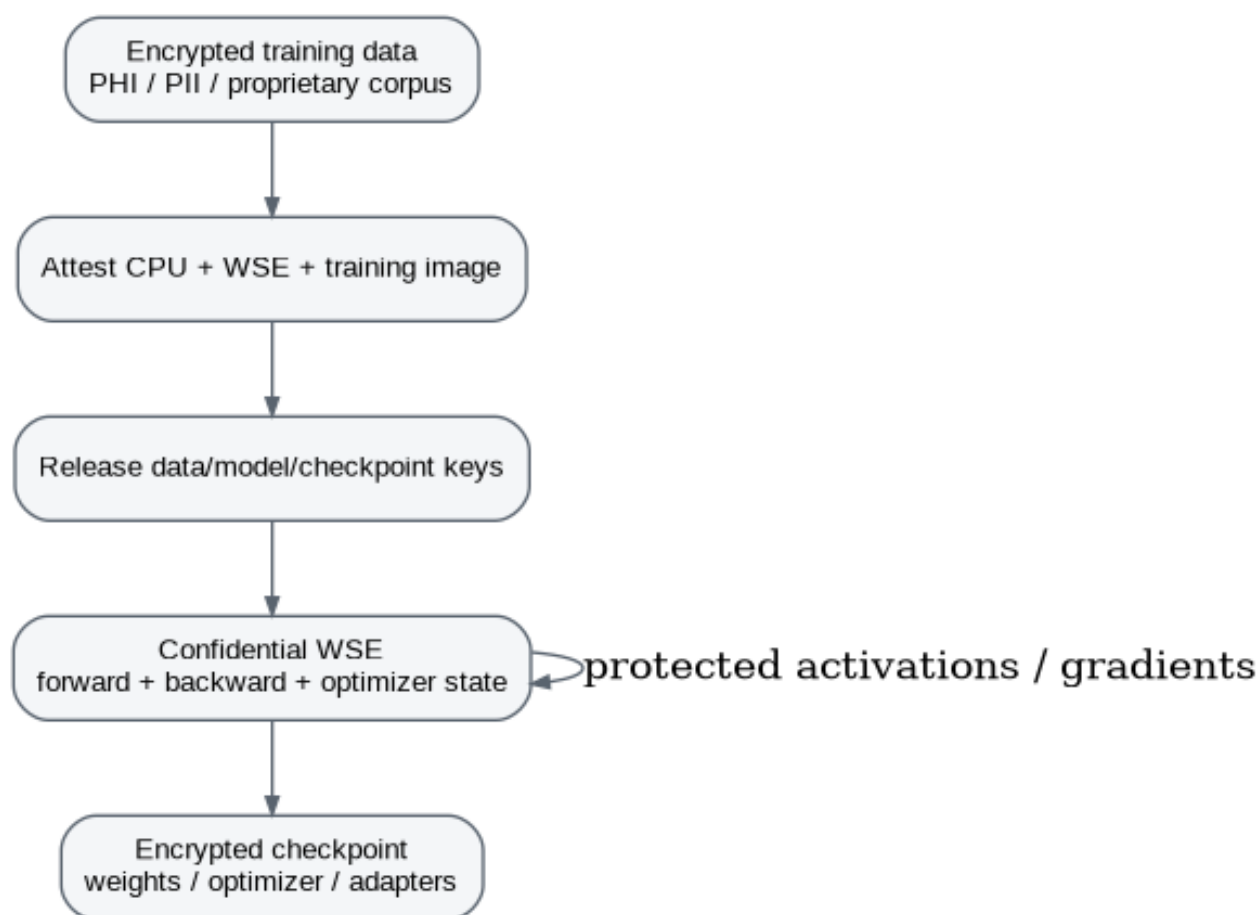


Figure 8. Confidential training/fine-tuning flow. Training data, model state, optimizer state and checkpoints can be kept inside the same attested security policy.

For LoRA/adaptor fine-tuning, a particularly useful deployment model is to keep the base model encrypted and controlled by the model owner while allowing the customer to provide an encrypted private dataset and receive an encrypted adaptor/checkpoint. The service can then attest the complete training environment without giving the infrastructure operator plaintext access to either party's protected material.

For full fine-tuning, the operational envelope expands because optimizer state and intermediate checkpoints can be large. The platform should therefore distinguish "confidential compute" from "confidential storage." The former protects tensors while the job is running; the latter seals artifacts when they leave the protected domain. KMS-issued per-job keys and an authenticated checkpoint format can bridge the two.

Multi-party training is an especially important extension. Healthcare, finance and sovereign workloads may require multiple parties to contribute data while the model owner and infrastructure operator remain mutually untrusted. Remote attestation allows each participant to verify that the other party's data will be processed only by the approved software configuration. That use case is directly aligned with the collaborative confidential-ML pattern described by IETF RATS and with the high priority healthcare organizations placed on privacy-preserving collaboration in the 2025 CCC study.¹⁸¹⁹

11. Rust and Mojo software architecture

The software proposal should be modern without pretending that Python can disappear from the existing ecosystem overnight. Cerebras currently exposes PyTorch, Model Zoo and inference APIs, and its platform compiler converts model definitions into WSE-executable programs. The practical approach is to keep Python/PyTorch as high-level

18 Confidential Computing Consortium, "2025 industry study / adoption findings."

19 IETF RFC 9334, "Remote ATtestation procedureS (RATS) Architecture."

compatibility and orchestration interfaces while introducing Rust and Mojo as the security-critical and performance-critical layers.²⁰

Rust is especially well suited to the security control plane. rustc performs high-value analyses in MIR, including borrow checking and dataflow checks, then lowers MIR to LLVM IR for code generation. That architecture is relevant because a security-sensitive runtime benefits from strong compile-time safety while still targeting LLVM-supported machine code. It also makes Rust a pragmatic choice for a portable control plane that can run in x86 confidential VMs and on embedded security processors.²¹

Mojo is particularly interesting for the accelerator side because it was designed around MLIR and heterogeneous hardware. Mojo exposes MLIR directly enough to express hardware-specific operations and can lower through LLVM-level representations. The language documentation explicitly positions it for high-performance AI infrastructure and extensibility to new hardware backends.²²

The important technical correction is that Rust and Mojo do not share one identical intermediate-representation chain. Rust uses MIR and then LLVM IR; Mojo is MLIR-centered and can lower through LLVM. C-CCA should exploit this complementarity rather than manufacture a false common compiler story. A stable secure ABI and explicit security annotations are more important than forcing both languages through the same front end.

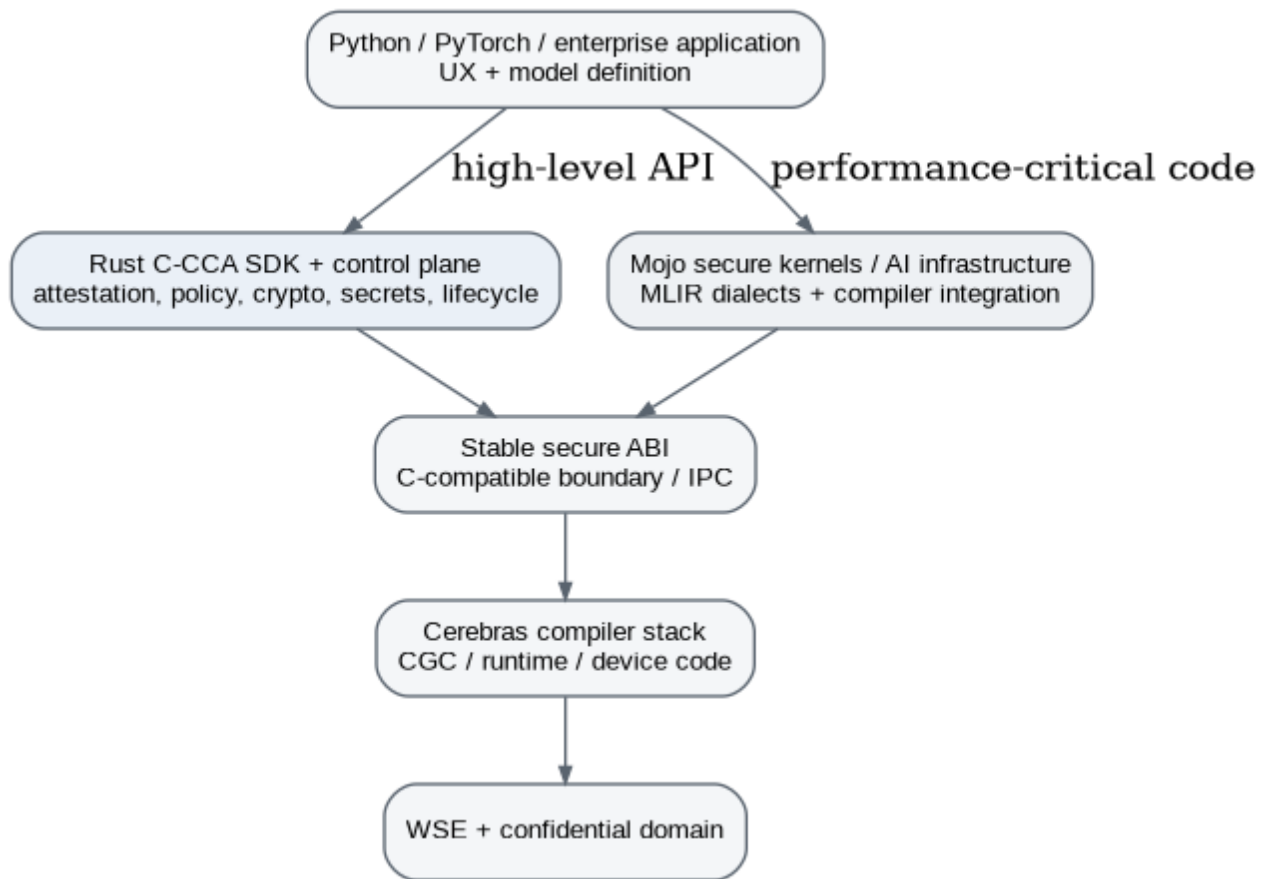


Figure 9. Proposed software stack. Python/PyTorch remains a compatibility and UX layer; Rust owns security control; Mojo owns performance-critical AI/device code; both meet at a stable secure ABI and the Cerebras compiler/runtime stack.

Layer	Preferred language	Primary responsibilities	Security role
Application / model authoring	Python/PyTorch	Model definition, service integration, business logic	Outside minimal TCB where possible
Confidential control plane	Rust	Attestation, key release, policy, lifecycle, IPC, secure orchestration	High-assurance TCB candidate

²⁰ Cerebras, “CSoft / Software Platform” and Training API documentation.

²¹ Rust Compiler Development Guide, “Overview of the compiler.”

²² Modular, “Mojo Manual” and “Mojo inline MLIR reference.”

Layer	Preferred language	Primary responsibilities	Security role
Crypto service	Rust + hardware primitives	Key handles, crypto operations, zeroization	Small TCB
AI/device kernels	Mojo	High-performance tensor/graph kernels, secure buffer operations	Measured workload code
Compiler / lowering	Cerebras compiler + MLIR/LLVM where applicable	Graph compilation, mapping, code generation	Part of measured workload
Legacy/native integration	C/C++ ABI where needed	Drivers, existing libraries, hardware vendor interfaces	Compatibility surface; minimize TCB

12. API and compiler integration proposal

The software API should expose security semantics as first-class objects rather than low-level ioctl-style commands. Recommended core objects are ConfidentialDomain, AttestationEvidence, AttestationPolicy, KeyHandle, ProtectedBuffer, SecureSession, WorkloadManifest and ConfidentialJob. Each object should be bound to an explicit owner, lifetime and policy.

A job-submission sequence can then become deterministic:

1. Create a signed WorkloadManifest containing runtime, model and policy measurements.
2. Create or allocate a confidential WSE domain and bind the workload image.
3. Establish the CPU CVM-to-WSE secure session and obtain composite evidence.
4. Send evidence to the tenant-controlled verifier and receive an Attestation Result.
5. Ask the KMS/HSM to release the model/data keys only if policy matches.
6. Load encrypted weights and sensitive data through protected buffers.
7. Execute inference or training inside the confidential domain.
8. Export only policy-approved ciphertext/results and seal checkpoints.
9. Destroy the domain, invalidate operational key lineage and scrub state.

Mojo should expose secure buffer and domain capabilities through a small standard library layer. A secure tensor should carry ownership metadata that is visible to the runtime and compiler but is not directly forgeable by normal application code. Examples include ProtectedTensor, AttestedDevice, SecureSession and KeyHandle wrappers. The compiler can use these abstractions to reject accidental copies from protected memory to ordinary host-visible buffers when such a transfer violates policy.

The most ambitious compiler integration is to add a C-CCA MLIR dialect. This dialect would represent operations such as `secure_domain_enter`, `protected_alloc`, `secure_copy`, `key_handle_use`, `attested_submit` and `protected_zeroize`. These operations could lower to the Cerebras compiler/runtime primitives. The key benefit is not cryptographic implementation in MLIR itself; it is making security-relevant data movement explicit in the intermediate representation so the compiler cannot accidentally erase the security boundary during optimization.

Rust can integrate through a stable C ABI or an explicit RPC/IPC boundary when the compiler stack remains native. Mojo already provides C FFI facilities, so a Rust-produced security library can be surfaced to Mojo without requiring a C/C++ application layer to own the security logic.²³

```
// Conceptual Mojo API
fn confidential_submit(job: WorkloadHandle,
                     session: SecureSession,
                     input: ProtectedTensor) raises -> ProtectedTensor:
    # Runtime verifies policy and submits only over the attested channel
    ...

// Conceptual Rust control-plane API
let evidence = tee.attest(&manifest)?;
let result = verifier.appraise(&evidence, &policy)?;
let model_key = kms.release(&result, KeyPurpose::Model)?;
let session = wse.open_secure_session(&result)?;
job.load_encrypted_weights(model_key, session)?;
```

²³ Modular, “ffi” documentation for Mojo interoperability.

13. Multi-WSE, MemoryX, SwarmX, and cluster confidentiality

A Cerebras confidential workload must scale with the architecture. CS-4 itself combines three WSE-3T processors, and Cerebras describes future scaling through modular rack-scale infrastructure. The security design should therefore treat a rack-scale system as a composite device rather than a flat collection of independent TEEs.²⁴²⁵

Composite attestation should include each WSE, the supporting CPU CVM, relevant I/O modules, MemoryX state and the cluster configuration that determines workload membership. The final cluster measurement can be a deterministic aggregate of per-component evidence plus topology and policy. This prevents a malicious operator from replacing one WSE or altering the cluster membership without changing the attested identity.

SwarmX requires special care because training can broadcast weights and reduce gradients across WSEs. A protected workload should use authenticated domain membership for those flows. A weight broadcast should be accepted only by WSE domains whose attestations satisfy the same workload policy, and a gradient reduction should be scoped to the intended training job. Otherwise, a compromised management plane might inject a foreign participant into a confidential training job.

MemoryX should be viewed as a cryptographic storage endpoint, not merely as external RAM. Its data should remain encrypted under job/model keys, with the decryption session established only after attestation. If the MemoryX control path itself is not part of the hardware TEE boundary, the system should design the data plane so operators can observe traffic metadata but cannot obtain the plaintext model state.

14. Side-channel, rollback, and operational security

A TEE is not automatically side-channel safe. A wafer-scale accelerator creates unusual channels: fabric contention, queue occupancy, timing, thermal behavior, power draw, performance counters, scheduling, memory access patterns and model-dependent traffic. The system should therefore document which channels are in scope for each security profile.

For high-assurance confidential mode, disable or restrict performance counters and tracing features that can expose tenant-specific execution characteristics. NVIDIA's H100 confidential mode is a concrete precedent: its full CC-On mode activates security controls and disables performance counters to reduce side-channel opportunities.²⁶

Rollback protection is equally important. A valid attestation of an old but vulnerable image is still a security failure. Security-version counters, firmware revocation lists, minimum-version policies and workload-specific reference values should therefore be part of the attestation result and KMS policy.

Confidential state migration should be treated cautiously. If a domain is suspended or moved, the destination must prove an acceptable security state before the source releases the sealed state key. The default high-assurance rule should be "no migration unless the destination is attested and policy-compatible."

Debug and break-glass operations must be separated from normal administration. A production confidential domain should reject raw memory inspection and arbitrary device debug. Any exceptional debug mode should require explicit attested authorization, should generate auditable evidence, and should transition the workload into a distinct security state that makes key release more restrictive or impossible.

15. Performance and implementation considerations

The biggest risk to adoption is not whether the architecture is technically possible; it is whether security features compromise the latency and throughput that make Cerebras attractive. The design should therefore avoid unnecessary encryption/decryption passes and expensive per-packet cryptography where hardware trust metadata can be carried at low cost.

A sensible hardware objective is to protect boundaries, not every internal arithmetic operation. Once data is inside a protected domain, the accelerator may operate on plaintext tensors internally; the security hardware needs to prevent unauthorized observers from obtaining that plaintext. Encryption is most valuable at domain crossings: SRAM line

24 Cerebras Systems, "Cerebras Unveils CS-4: Up to 30 Times Faster than GPU-based Solutions" (18 Aug 2026).

25 Cerebras, "Ultrafast Frontier Inference | Cerebras Hot Chips 2026" (25 Aug 2026).

26 NVIDIA, H100 confidential-computing operation modes and security controls.

fills/evictions where applicable, external I/O, MemoryX transfers, protected fabric crossings, checkpoint storage and host-visible buffers.

A proposed performance budget for early prototypes is an end-to-end inference overhead target below roughly 2-5% for high-throughput workloads, while measuring separately the one-time costs of attestation and key establishment. This is a research target, not a documented Cerebras performance commitment. The actual target must be determined by workload profiling on the WSE architecture.

Cost center	Likely overhead source	Mitigation
Attestation	Evidence collection / signature verification	Cache attestation result with short policy-bound lease
Key release	Network round-trip to KMS/HSM	Session key derivation inside TEE; batch key authorization
Memory protection	Encryption/integrity per protected transaction	Wide parallel crypto engines; metadata locality
Fabric security	Domain checks + tags	Hardware ACL lookup adjacent to routers
I/O protection	Authenticated/encrypted DMA	Dedicated engines and large-buffer operations
Compiler security	Protected buffer annotations/checks	Static analysis + explicit secure MLIR dialect
Debug restrictions	Reduced observability	Secure counters exposed only through attested telemetry

16. Roadmap and validation program



Figure 10. Proposed implementation roadmap. The security plane can be prototyped around current systems before the new WSE silicon is required.

Phase 0 — Architecture and threat model. Freeze the security invariants, trust boundaries, attacker classes, attestation claims, key-release policy, and minimum hardware primitives. Define the security certification target so that later implementation choices remain measurable.

Phase 1 — Software prototype on existing Cerebras infrastructure. Run the Cerebras runtime and model-serving control plane inside a TDX or SEV-SNP CVM; use a customer-controlled KMS/HSM; build the Rust attestation/secret broker; protect model artifacts and test that no raw key or sensitive prompt is exposed to the host software. This phase demonstrates the business case without new WSE silicon.

Phase 2 — Device composition prototype. Implement a CPU-CVM-to-WSE secure-session shim and a workload measurement model. Even if the existing WSE cannot provide hardware confidentiality, the prototype can measure the exact data flows that a future secure WSE must absorb into the trusted boundary.

Phase 3 — Future WSE silicon. Add the WSE Security Manager, root of trust, secure boot, confidential domain manager, protected SRAM, secure DMA and fabric access controls. The acceptance test is not a marketing claim; it is an independently verifiable statement of the six security invariants.

Phase 4 — Confidential rack and cluster. Extend the design through MemoryX, SwarmX and rack-scale I/O. Implement composite attestation and multi-WSE workload identity. Test recovery, failover, reset, revocation and domain destruction across realistic production operations.

Phase 5 — Productization. Expose C-CCA as deployment profiles, customer-managed attestation policies, managed KMS integrations, confidential model packaging, confidential agent credentials, and sovereign deployment controls. Publish a transparent security target and attestation claim schema so customers can independently verify the platform.

Milestone	Demonstration	Exit criterion
Software PoC	Confidential LLM inference in CVM with key release	No plaintext model/keys in host; verifiable attestation

Milestone	Demonstration	Exit criterion
System PoC	CVM ↔ WSE protected session + measurement	Defined WSE trust boundary and measurable data flows
Silicon prototype	WSE confidential domain	Hardware rejects cross-domain access
Cluster prototype	Composite multi-WSE attestation	KMS can bind secret release to whole workload
Production readiness	Healthcare/finance/sovereign pilot	Customer verifies confidentiality properties independently

17. Demand and high-value application domains

The strongest use cases are those in which data confidentiality, model IP, and action authority intersect. That intersection is particularly strong for sovereign/classified AI, healthcare and pharma, financial services and trading, and autonomous enterprise agents. The 2025 CCC study reported financial services as the highest full-production industry segment among respondents, followed by healthcare and government, while Cerebras already markets AI solutions into finance, healthcare/pharma, government/sovereign settings, and fast agentic inference.²⁷²⁸²⁹³⁰

Domain	Protected assets	Why Cerebras + C-CCA is compelling
Sovereign / classified AI	National data, local models, mission data, secrets	Need verifiable isolation from infrastructure operators; low latency helps mission workflows
Healthcare / pharma	PHI, genomics, research corpora, proprietary models	Allows sensitive analytics/training while limiting exposure to operators and collaborators
Finance / trading	Transactions, research, strategies, customer data, signing keys	Confidential models plus non-exportable signing authority and fast decision loops
AI agents	OAuth/API keys, wallets, databases, private RAG, user state	Credentials can be released to attested code with narrow capabilities
Cybersecurity	Incident data, telemetry, detection models, response credentials	Sensitive telemetry can remain confidential while inference stays extremely low latency
Blockchain / threshold crypto	Private keys, MPC shares, signing policies	TEE can provide last-resort plaintext residency and policy enforcement
Defense / intelligence	Classified imagery, signals, mission models	Supports cryptographically verifiable execution in controlled infrastructure
Enterprise confidential AI	Internal documents, contracts, legal and HR data	Reduces trust required in multi-tenant or outsourced AI infrastructure

An especially differentiated use case is the confidential AI agent: the LLM, RAG state, policy engine and credential broker execute inside the composite TEE, while external systems receive only authenticated tool calls and encrypted results. The key security rule is that credentials are released to attested code under a policy, not handed to a machine or administrator.

18. Standards and interoperability strategy

C-CCA should reuse established standards wherever they map cleanly, while defining Cerebras-specific claims and hardware mechanisms where the architecture is novel. IETF RATS provides the attestation architecture and claim-evaluation model. SPDH provides a mature device-authentication and measurement-session vocabulary. PCI-SIG

²⁷ Confidential Computing Consortium, “2025 industry study / adoption findings.”

²⁸ Cerebras, “What Is Sovereign AI—and How Cerebras Helps Nations” (26 May 2026).

²⁹ Cerebras, “AI for financial services.”

³⁰ Cerebras, “Health & Pharma.”

TDISP is a reference point for extending virtualization-based TEEs to devices. TPM 2.0 remains relevant to platform identity/attestation infrastructure on the CPU side.^{31,32,33,34}

The key is to avoid inventing a proprietary end-to-end trust protocol when interoperability can be achieved at the evidence and device-session layers. Cerebras can define a WSE-specific attestation profile and a C-CCA workload manifest while still emitting evidence and results in forms that standard RATS tooling can understand.

The system should also expose customer-verifiable claims in a stable format. Useful claims include device model, security mode, firmware measurement, minimum security version, confidential-domain state, model/runtime measurement, protected-memory state, device identity, cluster composition and policy version. These claims should be signed and bound to freshness and challenge data.

19. Strategic recommendation to Cerebras

The strategic recommendation is to position confidential computing as the next architectural layer of the Cerebras platform rather than as a separate security appliance. Cerebras has already built a differentiated hardware/software co-design around wafer-scale compute. A wafer-native confidential domain would extend that co-design into a part of the AI stack where software-only competitors cannot easily follow.

The product message should be precise: "fast AI" becomes "trusted AI at wafer scale." The value proposition is not that Cerebras merely encrypts storage, but that customers can obtain cryptographically verifiable evidence that their model, data and cryptographic authority are running inside an approved execution boundary that excludes the infrastructure operator from plaintext access.

The executive program should be a cross-functional architecture sprint spanning hardware security, WSE architecture, systems, firmware, runtime/compiler, cloud infrastructure and product security. The first output should be a security target and working proof of attested inference; the second should be a WSE hardware requirements specification; the third should be an external customer validation program in one or two high-value markets such as healthcare, financial services or sovereign AI.

This aligns with the direction of Cerebras' current platform development. Cerebras has publicly emphasized security engineering across the Linux kernel, firmware, secure boot, measured boot and attestation strategy, including work spanning the wafer-scale system and supporting host nodes. That is not evidence that WSE-3T already provides a complete hardware TEE; it is evidence that the security architecture is a live engineering concern and therefore a timely surface for a deeper platform proposal.³⁵

20. Research questions and next steps

The hardest technical question is the minimum silicon required to make the WSE a genuine confidential device without sacrificing the architectural advantages of wafer scale. That question should be attacked empirically through area, power and latency budgets rather than through a direct copy of SGX or GPU CC designs.

A second research question is the best granularity of confidential domains. Large domains reduce metadata overhead and simplify scheduling; smaller domains improve isolation and multi-tenancy. A useful first target is one domain per protected workload or protected tenant partition, with optional subdivision for a crypto service or agent credential service.

A third question is the optimal memory-integrity construction for distributed SRAM and fabric traffic. This should be evaluated with the WSE bandwidth profile, fault model, access pattern, and expected side-channel protections. The architecture should support an integrity mechanism with bounded memory overhead and parallel verification.

A fourth question is how far compiler enforcement should go. A secure MLIR dialect can encode protected buffers and security-sensitive operations, but the hardware must remain the ultimate enforcement point. Compiler checks are

31 IETF RFC 9334, "Remote ATtestation procedureS (RATS) Architecture."

32 DMTF, "Security Protocol and Data Model (SPDM)."

33 PCI-SIG, "TEE Device Interface Security Protocol (TDISP)."

34 Trusted Computing Group, "TPM 2.0 Keys for Device Identity and Attestation" (v1.10r9, Apr 2025).

35 Cerebras Systems, "Hardware / Low Level Security Engineer" role (2026).

defense in depth, not the root of confidentiality. Rust type safety can similarly reduce accidental leakage in the control plane but cannot substitute for hardware isolation.

Finally, the platform needs a measurable customer-verification story. The test should be reproducible by an external party: obtain attestation evidence, verify the chain of trust, compare the workload measurement to a published policy, request a key from a customer-controlled KMS, run the workload, and demonstrate that the untrusted host cannot retrieve model/data/key plaintext. That is the experiment that turns C-CCA from an architectural idea into a platform capability.

Appendix A. Security invariants

1. No secret release without valid, fresh attestation that satisfies the configured policy.
2. No host or hypervisor access to confidential execution memory through ordinary privileged interfaces.
3. No plaintext model weights, sensitive activations, KV cache or protected training state leave the trusted domain unless explicitly authorized by policy.
4. No private-key or threshold-share export from the cryptographic service; applications operate through non-exportable capabilities or handles.
5. No cross-domain memory, fabric or DMA access without a hardware-enforced authorization rule.
6. Destroying a confidential domain destroys or invalidates its operational key hierarchy and renders residual state unusable.
7. Production debug of an active confidential domain is disabled or requires independently attested authorization.
8. Rollback to a disallowed firmware/runtime/security version cannot produce a successful attestation result.

Appendix B. Threat matrix

Threat	Protected by	Residual consideration
Malicious hypervisor	CPU CVM memory encryption/isolation + attestation	Metadata and availability channels remain possible
Host memory scraping	CVM + WSE protected memory	Device interface must be protected too
Malicious driver	Secure device session + command authentication	Driver can still attempt DoS
Unauthorized DMA	Secure DMA / IOMMU-like hardware policy	Must cover every device endpoint
Firmware substitution	Secure/measured boot + version policy	Supply-chain endorsements must be maintained
Cross-domain WSE access	CDID + fabric ACL + memory authorization	Physical side channels require separate analysis
Model theft	Encrypted model + attested key release	Authorized workload can still inspect its own model by design
Private-key extraction	Non-exportable handles + protected memory + zeroization	Side channels and compromised TCB remain concerns
Replay/rollback	Counters, epochs, integrity metadata, version policies	Recovery semantics need careful design
Debug leakage	Debug gate + auditable authorization	Emergency workflows can reduce security state
Fabric traffic inference	Authenticated traffic + scheduling isolation	Traffic analysis may still leak metadata
Physical attack	Optional high-assurance profile	May require package-level countermeasures beyond standard TEE

Appendix C. Terminology

C-CCA: Cerebras Confidential Computing Architecture, the proposed composite TEE platform.

CVM: Confidential Virtual Machine implemented with a CPU technology such as Intel TDX or AMD SEV-SNP.

WSE Confidential Domain: A hardware-enforced spatial security domain spanning selected WSE processing elements, SRAM and fabric endpoints.

CDID: Confidential Domain ID; hardware metadata identifying domain ownership and access rights.

Cerebras RoT: Proposed WSE hardware root of trust supporting identity, secure boot and key derivation.

MemoryX: Cerebras technology for external model-weight storage/streaming in supported execution modes.

SwarmX: Cerebras interconnect technology connecting multiple WSE systems for distributed training/execution.

RATS: IETF Remote Attestation procedureS architecture for evidence, verification and policy-based trust decisions.

SPDM: DMTF Security Protocol and Data Model for device authentication, measurement and secure sessions.

TDISP: PCI-SIG TEE Device Interface Security Protocol for trusted device interfaces in virtualization-based TEEs.

KMS/HSM: External authority for durable key protection and policy-controlled secret release.

Workload measurement: Cryptographic measurement of the software, configuration and model identity that a confidential workload will execute.

Sources

1. Cerebras Systems, "Introducing Cerebras CS-4: The Fastest AI Just Got Faster," 18 Aug 2026. <https://www.cerebras.ai/blog/introducing-cerebras-cs-4>
2. Rust Compiler Development Guide, "Overview of the compiler," current documentation. <https://github.com/rust-lang/rustc-dev-guide/blob/main/src/overview.md>
3. Modular, "Mojo Manual" and "Mojo inline MLIR reference," current documentation. <https://docs.modular.com/mojo/manual/> ; <https://docs.modular.com/mojo/reference/mojo-inline-mlir/>
4. Intel, "Intel Trust Domain Extensions (Intel TDX)," current documentation. <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html>
5. AMD, "Confidential Computing Starts with AMD EPYC Server CPUs," current documentation. <https://www.amd.com/en/products/processors/server/epyc/confidential-computing.html>
6. Microsoft Learn, "Confidential AI - Azure Confidential Computing," current documentation. <https://learn.microsoft.com/en-us/azure/confidential-computing/confidential-ai>
7. Confidential Computing Consortium, "2025 industry study / adoption findings," 2025. <https://confidentialcomputing.io/2025/>
8. Cerebras Systems, CS-4 announcement, discussion of agentic inference, 18 Aug 2026. <https://www.cerebras.ai/blog/introducing-cerebras-cs-4>
9. Cerebras Developer Documentation, "Overview / Cerebras Wafer-Scale cluster," current documentation. <https://training-api.cerebras.ai/en/latest/wsc/Concepts/how-cerebras-works.html>
10. Cerebras Systems, "Cerebras Unveils CS-4: Up to 30 Times Faster than GPU-based Solutions," 18 Aug 2026. <https://investors.cerebras.ai/news-releases/news-release-details/cerebras-unveils-cs-4-30-times-faster-gpu-based-solutions>
11. Cerebras Systems, "Announcing the Cerebras Architecture for Extreme-Scale AI," architecture discussion of MemoryX and SwarmX. <https://www.cerebras.ai/blog/announcing-the-cerebras-architecture-for-extreme-scale-ai>
12. IETF RFC 9334, "Remote Attestation procedureS (RATS) Architecture," 2023, updated 2026. <https://datatracker.ietf.org/doc/rfc9334/>
13. DMTF, "SPDM," current standard information. <https://www.dmtf.org/standards/spdm>
14. PCI-SIG, "TEE Device Interface Security Protocol (TDISP)" and TDISP eXtended TEE extensions. <https://pcisig.com/PCI%20Express/ECN/Base/TEEDeviceInterfaceSecurityProtocol>
15. NVIDIA, "Confidential Computing on NVIDIA H100 GPUs for Secure and Trustworthy AI," technical blog and attestation workflow. <https://developer.nvidia.com/blog/confidential-computing-on-h100-gpus-for-secure-and-trustworthy-ai/>
16. RustCrypto, "zeroize" crate documentation. <https://docs.rs/zeroize/latest/zeroize/>
17. NVIDIA, H100 confidential-computing operation modes and security controls. <https://developer.nvidia.com/blog/confidential-computing-on-h100-gpus-for-secure-and-trustworthy-ai/>
18. Cerebras Developer Documentation, "Training and fine-tuning a Large Language Model (LLM)" and fine-tuning guides. <https://training-docs.cerebras.ai/rel-2.10.0/getting-started/fine-tune-your-first-model>
19. Cerebras, "CSoft / Software Platform" and Training API documentation. <https://www.cerebras.ai/product-software> ; <https://training-api.cerebras.ai/>
20. Modular, "ffi" documentation for Mojo interoperability. <https://docs.modular.com/mojo/std/ffi/>
21. Cerebras, "Ultrafast Frontier Inference | Cerebras Hot Chips 2026," 25 Aug 2026. <https://www.cerebras.ai/blog/ultrafast-frontier-inference-cerebras-deep-dive-at-hot-chips-2026>
22. Cerebras, "What Is Sovereign AI—and How Cerebras Helps Nations," 26 May 2026. <https://www.cerebras.ai/blog/what-is-sovereign-ai-and-how-cerebras-helps-nations>
23. Cerebras, "AI for finance services," current industry page. <https://www.cerebras.ai/industry-financial-services>
24. Cerebras, "Health & Pharma," current industry page. <https://www.cerebras.ai/industry-pharma>
25. Trusted Computing Group, "TPM 2.0 Keys for Device Identity and Attestation," v1.10r9, Apr 2025. <https://trustedcomputinggroup.org/resource/tpm-2-0-keys-for-device-identity-and-attestation/>

26. Cerebras Systems, "Hardware / Low Level Security Engineer" role, 2026, discussing secure boot, measured boot, attestation and hardware roots of trust across the wafer-scale system. <https://jobs.ashbyhq.com/cerebras/4e706d0d-65a6-4a36-9ff3-b9e9da16a618>

Evidence boundary. Public Cerebras documentation supports the current WSE/CS architecture, workload model, and the company's active security engineering direction. The WSE Confidential Domain, per-domain protected SRAM/fabric, WSE attestation claims, composite C-CCA workload identity, and associated Rust/Mojo security APIs are proposals in this report and should be treated as architecture targets for future hardware and software rather than as existing WSE-3T/CS-4 capabilities.